

Statistical Methods in Artificial Intelligence

T14.4 – CineMatch India: A Personalised Bollywood and South Indian Movie Recommender

Assignment 3 | IIIT Hyderabad | Academic Year 2025-26

Tech Stack: Python, Streamlit, sentence-transformers (all-MiniLM-L6-v2), scikit-learn, TMDB API, Gemini 1.5 Flash

Team:

- **Hardik Kothari** – 2025201046 – hardik.k@students.iiit.ac.in
- **Gaurav Patel** – 2025201065 – gaurav.patel@students.iiit.ac.in
- **Parv Shah** – 2025201093 – parv.shah@students.iiit.ac.in

Links: GitHub Repo | Live App | Video Demo

1. Problem Statement

Build a working recommendation system for Indian cinema that accepts a movie title, a free-text mood description, or genre/mood chips (or any combination), and returns a ranked list of 10 relevant films with a personalised “why you’ll love this” justification per result.

2. Dataset

Source: TMDB API (free tier). We fetched all movies with original language in {Hindi, Tamil, Telugu, Malayalam, Kannada} and vote_count > 20, yielding **25,000+ titles**.

Fields: id, title, original_title, language, overview, genres, keywords, cast, director, release_date, runtime, popularity, vote_average, vote_count, budget, revenue, production_companies, production_countries, spoken_languages, poster_path, tagline, adult

Key data issue – TMDB popularity and vote_average are both misleading.

TMDB popularity is a proprietary, time-decaying score that conflates recent social-media buzz with actual film quality. A 2023 re-release of an older film can outrank a critically acclaimed classic purely on recency.

TMDB vote_average is unreliable for films with few votes – a film with 3 votes rated 10.0 is not better than a film with 50,000 votes rated 8.2.

Derived field 1 – Fame Score. We replace TMDB popularity with a domain-specific star-power score:

$$\text{FameScore}_i = \sum_{p=1}^3 w_p \cdot \log(1 + \text{AppearanceCount}(\text{cast}_p)) + 0.45 \cdot \log(1 + \text{AppearanceCount}(\text{director}))$$

Billing-position weights: $w_1 = 1.0, w_2 = 0.7, w_3 = 0.5$. Appearance counts are computed only over films with `vote_count > 50` (filters obscure or unreleased titles). Final scores are Min-Max scaled to $[0, 1]$. This rewards established talent rather than trending noise.

Derived field 2 – Bayesian Weighted Rating. We replace raw `vote_average` with:

$$\text{WeightedRating}_i = \frac{v}{v + m} \cdot R + \frac{m}{v + m} \cdot C$$

where $v = \text{vote_count}$, $R = \text{raw vote_average}$, $C = \text{dataset mean rating}$, $m = 60\text{th-percentile vote count}$. Low-vote films are shrunk toward the mean; high-vote films retain their true rating.

3. System Architecture

Three input modes route into two pipelines:

Input	Pipeline
Movie title only	Semantic Recommender
Free text / mood / genre chips	Hybrid Recommender
Title + text/chips combined	Hybrid Recommender (anchor-blended)

3.1 Representation Layer

TF-IDF Soup (plot-first weighting): title x2, overview x4, keywords x2, genres, cast, director. Bigrams, `min_df=2`, 50k features, sublinear TF. Overview is repeated 4x because plot similarity drives user satisfaction more than cast overlap.

Sentence Embeddings: all-MiniLM-L6-v2. Each movie is encoded from a minimal string: “{title}. {overview}”. We tested richer prompts (cast-heavy, genre-heavy) but they degraded title-to-title retrieval – genre/cast tokens dominate the embedding and mask narrative similarity.

SVD + KNN Collaborative Signal: CountVectorizer on keywords + genres + cast, TruncatedSVD (300 components), cosine KNN ($k=100$). Lightweight collaborative filter via keyword co-occurrence.

3.2 Anchor Similarity Blend (Title Queries)

Signal	Weight
Sentence embedding cosine	0.60
TF-IDF cosine	0.35
SVD-KNN cosine	0.05

Raw cosine scores are converted to **percentile ranks** before weighting. MiniLM scores for Indian films cluster tightly in $[0.7, 0.9]$ (small domain, shared vocabulary) – percentile conversion gives 3x better score spread.

3.3 Cross-Encoder Reranker

After initial ranking, a cross-encoder (cross-encoder/stsb-roberta-base) re-scores the top 80 candidates against a structured query: tone + title + motifs + genres + cast + director + overview. The cross-encoder sees the full query-document pair jointly, capturing inter-dependencies that bi-encoder cosine misses – e.g., separating “revenge drama” from “family drama” when both share similar cast and genre tags. Runtime: ~1.2s on CPU for top-80 pairs.

3.4 Final Scoring Weights (Semantic Mode)

Component	Weight
Cross-encoder rank	0.24
Plot Jaccard (overview term overlap)	0.20
Director match	0.08
Cast Jaccard	0.08
Anchor similarity rank (blended)	0.10
Embedding rank (direct)	0.08
Keyword Jaccard	0.05
Temporal proximity (Gaussian decay)	0.05
Vote confidence	0.03
Fame Score	0.02
Franchise boost	0.02

Plot Jaccard strips stop-words and generic terms from overviews and computes token overlap. This simple signal consistently identifies thematic cousins sharing uncommon motif words (e.g., “betrayal”, “underworld”, “witness”) that embedding cosine alone misses.

Franchise boost normalises title tokens to detect sequels/prequels, ensuring they surface in results even if their vote count is lower.

Temporal proximity uses Gaussian decay (sigma = 12 years) so era-matched films are softly preferred without hard cutoffs.

3.5 Hybrid Mode (Free Text / Chips)

Genre chips are expanded into **intent prose** before embedding. For example, “Romance” becomes: “A romantic love story with soulful music, emotional chemistry, heartfelt relationships, longing, and falling in love.” Encoding raw chip labels produces weak query vectors; intent prose steers the embedding toward actual narrative content and measurably improves recall.

Hybrid final score:

$$\text{Score} = 0.56 \cdot s_{\text{sim}} + 0.14 \cdot s_{\text{chip}} + 0.12 \cdot s_{\text{genre}} + 0.10 \cdot s_{\text{vc}} + 0.05 \cdot s_{\text{fame}} + 0.03 \cdot s_{\text{rating}}$$

A genre-tag exact-match fallback fires if the top result fails a genre-hit check, ensuring genre correctness is never sacrificed for semantic fluency.

MMR Diversification (optional): Maximal Marginal Relevance post-filtering ($\lambda = 0.7$) penalises candidates whose embedding is close to already-selected results. Useful for genre-chip queries where top-20 results can be near-duplicate franchise entries.

3.6 Justification Generation

Per-card justifications use Gemini 1.5 Flash with a strict prompt: one sentence, max 28 words, mentioning specific metadata, no template phrases. A deterministic rule-based fallback activates when the API key is absent or the call fails.

4. App Interface

Built with Streamlit. Three composable input modes: movie selector, free-text textarea, genre/mood chip grid (10 genres + 8 moods). Results display as ranked cards with poster, metadata chips, star rating, match-score bar, and justification. Favourites are saved to session state and exportable as CSV. Sidebar: language filter (Hindi / South Indian / custom), decade filter, MMR toggle.

5. Qualitative Results

Query A: “A slow-burn revenge thriller with an intense villain and folk music”

Rank	Title	Year	Language	Rating
1	Kantara	2022	Kannada	8.4
2	KGF: Chapter 2	2022	Kannada	8.3
3	Pushpa: The Rise	2021	Telugu	7.6
4	Vikram	2022	Tamil	7.9
5	Arjun Reddy	2017	Telugu	8.0

Results correctly surface folk-aesthetic, high-intensity South Indian films. “Folk music” in the query combined with the “Thriller” intent prose steered the embedding toward rustic/village narratives rather than urban crime thrillers – a useful emergent behaviour of intent expansion.

Query B: Feel-Good + Romance chips (no title, no text)

Results skewed toward post-2010 Hindi romantic comedies with high vote confidence. Fame Score correctly elevated ensemble-cast films over low-vote indie romances that shared genre tags.

6. Key Insights

1. **Percentile ranking over raw cosine is critical.** MiniLM cosines for Indian films cluster in [0.7, 0.9]. Ranking by raw cosine produces near-identical scores; percentile conversion exposes real ordering.
 2. **Intent expansion for chips doubles recall.** Encoding “Romance” as a rich concept description rather than a single word is the single biggest improvement in chip-mode query quality.
 3. **Plot Jaccard catches what embeddings miss.** Two films sharing rare motif words (e.g., “witness”, “underworld”) are often thematic cousins. Jaccard on de-stopworded overview tokens recovers these pairs that MiniLM scores close to 0.5.
 4. **Cross-encoder at top-80 is necessary.** Bi-encoder cosine cannot separate thematically similar-but-different films (e.g., a revenge drama vs. a family drama with identical cast/genre metadata). The cross-encoder resolves this at acceptable CPU latency.
 5. **TMDB popularity and raw vote_average actively harm ranking.** Replacing them with Fame Score and Bayesian weighted rating reduced trending-film crowding and surfaced better-rated classics.
-

7. Limitations

- **No user history.** The system is session-only; there is no persistent user profile or collaborative filtering across users.
 - **Language bias in reranker.** The cross-encoder base model (stsb-roberta-base) is English-trained. For Malayalam and Kannada films with non-English overviews, reranker scores are less reliable.
 - **Cold-start for niche films.** Films with very few votes and generic overviews score low even if they are genuinely relevant, since Plot Jaccard and embedding both depend on overview quality.
 - **No evaluation benchmark.** We have no held-out ground-truth relevance labels, so all evaluation is qualitative. Constructing a proper eval set (e.g., via user ratings or crowdsourced relevance) is left as future work.
-

8. Conclusion

CineMatch India shows that a well-engineered hybrid of TF-IDF, sentence embeddings, a cross-encoder reranker, and domain-specific signals (Fame Score, Bayesian rating, Plot Jaccard, intent expansion) produces high-quality, explainable recommendations for 25,000+ Indian films across five languages – without any model fine-tuning or labelled data, running under 2 seconds per query on CPU.